

Course Machine Learning

Lesson #3 – Regression

**Professor
Abdelouahab Moussaoui**

1 — Simple Linear Regression

Reminder of the 4 essential steps in Supervised Learning

Remember, to solve a **learning problem supervised**, we saw that it is necessary to proceed in **4 steps** :

1. Have a **Dataset** (x, y) where x are the *features* and y is the *target*
2. Develop a **Model** whose **parameters** are to be found
3. Define a **Function Cost** that measures the error between the model and the y values of the Dataset
4. Use an **Algorithm Learning** which looks for the model (in reality the parameters) which minimize the Cost Function, that is to say which gives us the model with the smallest errors.

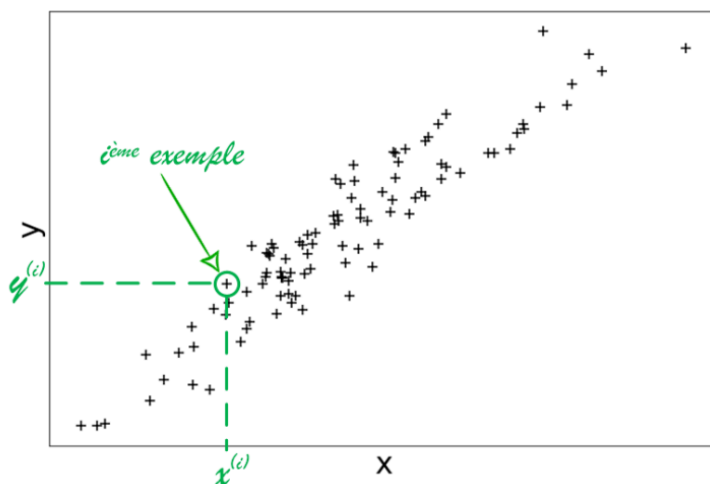
1. The Dataset of a Simple Linear Regression

Dans cet article, nous allons faire simple, dans le sens où nous allons voir comment développer un modèle de Machine Learning à partir d'un Dataset à une seule variable x_1 (que l'on notera simplement x). On aura donc un Dataset avec m exemples et $n=1$ « *feature* » variable.

y	x
$y^{(1)}$	$x^{(1)}$
$y^{(2)}$	$x^{(2)}$
$y^{(3)}$	$x^{(3)}$
...	...
$y^{(m)}$	$x^{(m)}$

To avoid remaining too abstract, we can very well imagine that this Dataset represents the price of an apartment y as a function of its living area x in a given city. This Dataset could give us the following point cloud:

Prix y en €	Surface x m^2
350,000	100
160,000	50
280,000	80
...	...
235,000	75

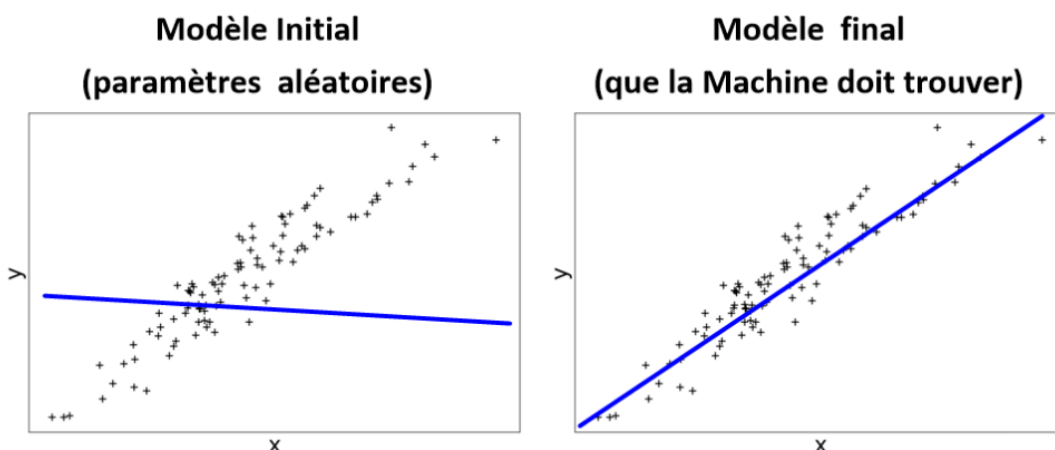


2. The Simple Linear Regression Model

Looking at our scatter plot, it clearly looks like it's following a **trend. linear**, that's why we're going to develop a **linear model** ! Our model: $f(x) = ax + b$.

Yes, yes! It's an affine function! Who would have thought that we could do machine learning with such simple math?!

At the moment we do not know the value of the **parameters** a and b , so it is impossible to draw a good straight line on the point cloud, unless you choose parameters at random. It will be the machine's role **to learn** these values by minimizing the cost function. Speaking of cost function, it's time to move on to step 3!



3. The Cost Function: The Mean Square Error

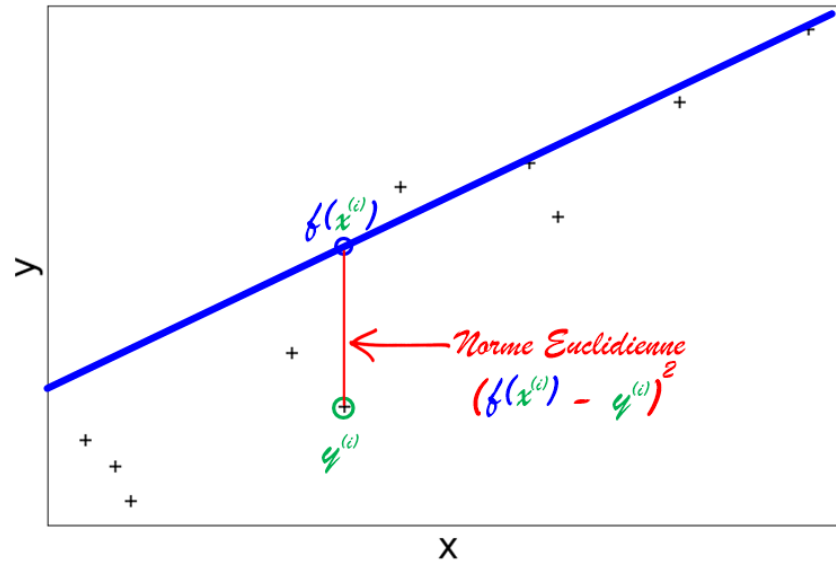
As a reminder, the cost function allows us to evaluate the performance of our model by measuring the errors between $f(x^{(i)})$ and $y^{(i)}$. The question we ask ourselves is: How can we measure these errors? I will reveal this to you with the following analogy.

Imagine viewing an apartment for €200,000. Your machine learning model predicts that this apartment is worth €300,000. You can conclude that your model is therefore making an error of $€300,000 - €200,000 = €100,000$.

So, we could say that to measure our errors, we must calculate the difference

$f(x) - y$. However, if your prediction $f(x)$ is less than y , then this error is negative, which is not practical.

So, to measure the errors between the predictions $f(x)$ and the values y of the Dataset, we calculate the square of the difference: $(f(x) - y)^2$. This is what we call the **norm Euclidean**, which represents the direct distance between $f(x)$ and in Euclidean geometry (the geometry we use every day)



For linear regression, the Cost function J will be the average of all our errors, that is:

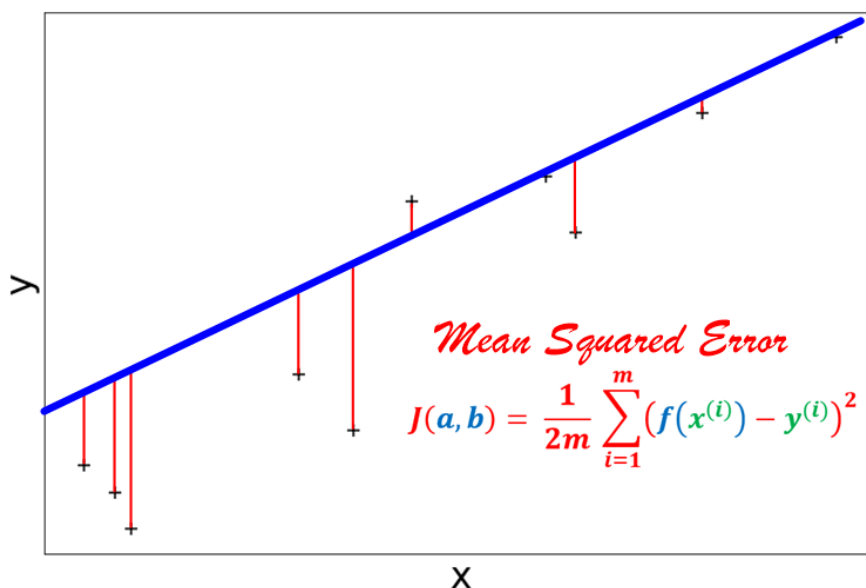
$$J = \frac{[f(x^{(1)}) - y^{(1)}]^2 + \dots + [f(x^{(m)}) - y^{(m)}]^2}{m}$$

By convention we write this function in the following way, by adding a coefficient $\frac{1}{2}$ to simplify a derivative calculation which will come later.

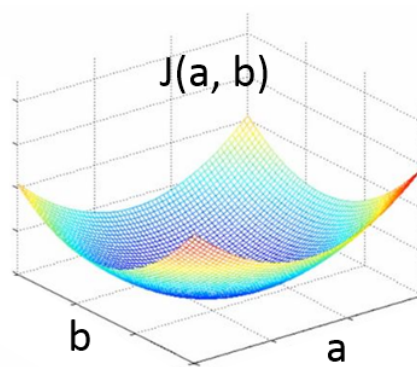
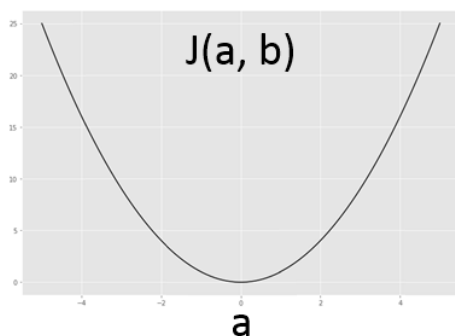
$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})^2$$

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (a \cdot x^{(i)} + b - y^{(i)})^2$$

This function has a name: in French we call it **Error Quadratic Average**, and in English it is called **Mean Squared Error**.



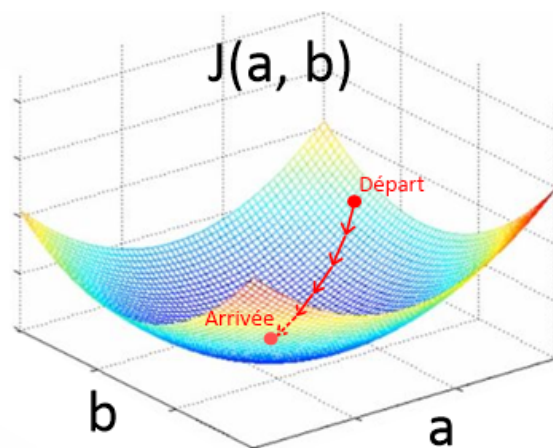
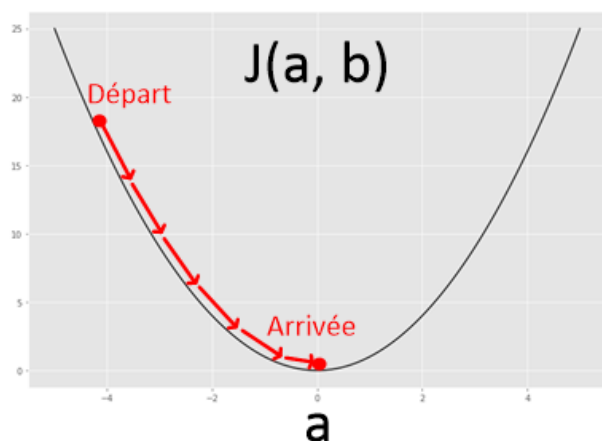
In the case of linear regression, this function depends on only two parameters: a and b . When we observe it, it has the appearance of a nice smooth valley. We say that it is a **convex function**. This property is very important to ensure convergence to the minimum with the descent **algorithm of gradient** that we will see in the next article.



4. The learning algorithm

As a reminder, we say that the machine learns when it finds which model parameters **minimize** the Cost function. We are therefore trying to develop a **minimization algorithm**. The good news is that there are a bunch of minimization methods (least squares method, Newton method, Gradient Descent, Simplex, etc.)

the least squares method is most often used (when the problem is simple) and the **Gradient algorithm Descent** (for more complicated regressions). Gradient Descent, which we will dissect in detail in the next article, allows you to gradually converge towards the minimum of any **convex function** (such as the Cost Function) by following the direction of the slope (the gradient) which descends, hence its name Gradient Descent.



Simple Linear Regression: In Conclusion

With linear regression, we have seen how to put into practice the 4 steps to solve a supervised learning problem. **This same mechanism** will be used to develop a speech recognition system, computer vision or chatbots. It will just be a matter of using another model with a different Cost function. In the next article I will explain in detail how the Gradient Descent algorithm works, which will allow us to complete our Machine Learning algorithm for linear regression, and finally be able to write our first Machine Learning program!

Note: Always write down your various **equations** and the **dimensions** of your Dataset on a sheet of paper before starting to program. This simple step can save you considerable time by avoiding bugs related to math errors.

2 — Why write equations in matrix form?

Right now, here is your linear regression program written **on course**. This program learns, thanks to *Gradient Descent*, which parameters a and b give the best model $f(x) = ax + b$.

On the other hand, this program is not at all practical to implement in this form.

To begin with, the calculation of Gradients $\frac{\partial J(a,b)}{\partial a,b}$ involves adding up the $x(i) \times (ax(i) + b - y(i))$ for all the values in our Dataset, **value after value**.

In the same vein, we can only use our model $f(x) = ax + b$ on a value $x^{(i)}$ at a time. So if you want to calculate millions of $f(x)$ predictions, you have to perform **this calculation millions of times**, for each desired $x^{(i)}$.

Want an aspirin? Don't take one, it's not worth it! Because we just need to reformulate our equations in **matrix form** to calculate the Gradient, the Cost Function, and the model on all the data in our Dataset **in one go**. Here's how to do it.

1. Transfer the Dataset into X, y matrices

In the presence of a Dataset (x, y) which contains m examples and n features, we transfer all the data $y^{(i)}$ into a vector Y of dimension $(m \times 1)$ and all the data $x^{(i)}$ into a matrix X to which we add a **bias column** (it is a column filled with "1"). The matrix X is therefore of dimension

$(m \times 1 + 1)$ and. In the case of linear regression with a single variable, we have $n = 1$ and therefore the matrix X is of dimension $(m \times 2)$ You will understand the usefulness of the **bias column** in a moment.

$$X = \begin{bmatrix} x^{(1)} & 1 \\ x^{(2)} & 1 \\ x^{(3)} & 1 \\ \vdots & \vdots \\ x^{(m-1)} & \vdots \\ x^{(m)} & 1 \end{bmatrix}$$

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ \vdots \\ y^{(m-1)} \\ y^{(m)} \end{bmatrix}$$

2. The $F=X.\theta$ model.

Currently, the linear model $f(x^{(i)}) = ax^{(i)} + b$ only allows one prediction to be calculated at a time, based on a single $x^{(i)}$. To solve this problem, simply create a vector F of dimension $(m \times 1)$ which contains all the predictions:

$$Y = \begin{bmatrix} f(x^{(1)}) \\ f(x^{(2)}) \\ f(x^{(3)}) \\ \vdots \\ f(x^{(m-1)}) \\ f(x^{(m)}) \end{bmatrix}$$

To calculate this vector, we use the following formula: $F = X.\theta$ Or $\theta = \begin{bmatrix} a \\ b \end{bmatrix}$ is the **vector parameter** that contains all the parameters of our model. We therefore have the matrix product:

$$F = X \cdot \theta = \begin{bmatrix} x^{(1)} & 1 \\ \dots & \dots \\ x^{(m)} & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax^{(1)} + 1 \times b \\ \dots \\ ax^{(m)} + 1 \times b \end{bmatrix} = \begin{bmatrix} f(x^{(1)}) \\ \dots \\ f(x^{(m)}) \end{bmatrix}$$

Note: the **bias** column of the matrix X therefore allows the matrix product X to be performed θ . We also say that the parameter b is a **bias parameter**.

3. The Cost Function $J(\theta)$

Currently, the cost function we have for linear regression is the mean square error:

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m (a \cdot x^{(i)} + b - y^{(i)})^2$$

We now know that $a \cdot x^{(i)} + b$ can be written in the form $X \cdot \theta$. Regarding $y^{(i)}$, we will create a vector Y which contains all the $y^{(i)}$ of the Dataset:

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

The vectors Y and $X \cdot \theta$ are both 2 of dimensions $(m \times 1)$, we can therefore perform the calculation $X \cdot \theta - Y$.

The rest is simple: we take the square of each element of the vector $X \cdot \theta - Y$, then we add up each of these elements, before dividing the whole by $\frac{1}{2m}$.

The matrix form of the cost function is then:

$$J(\theta) = \frac{1}{2m} \sum (X \cdot \theta - Y)^2$$

Note : J is neither a matrix nor a vector. It is a scalar (a number). Indeed, it would not make sense to put $J(a, b)$ in matrix form, because the cost function allows us to express the performance of our model with a **single value**. Ultimately, we could say that we are trying to write $J(a, b)$ as a matrix of dimension (1×1) , that is, a matrix containing only one number.

4. Gradients

To calculate gradients, we previously had to define a specific formula for each parameter.

$$\frac{\partial J(a, b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x^{(i)} \times (a \cdot x^{(i)} + b - y^{(i)})$$

$$\frac{\partial J(a, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (a \cdot x^{(i)} + b - y^{(i)})$$

It is possible to combine the calculation of these two gradients into a gradient vector $\frac{\partial J(a, b)}{\partial \theta}$:

$$\frac{\partial J(a, b)}{\partial \theta} = \begin{bmatrix} \frac{\partial J(a, b)}{\partial a} \\ \frac{\partial J(a, b)}{\partial b} \end{bmatrix}.$$

Note : this vector is therefore of dimension (2×1) or $(n+1 \times 1)$

To obtain this vector we write the following formula:

$$\frac{\partial J(a, b)}{\partial \theta} = \frac{1}{m} X^T \cdot (X \cdot \theta - Y)$$

5. Gradient Descent

As a reminder, the Gradient Descent algorithm updates the parameters a and b of our model iteratively with 2 lines of calculations:

[image coming soon]

But now that we have created a **parameter vector** $\theta = \begin{bmatrix} a \\ b \end{bmatrix}$ and a **gradient vector** $\frac{\partial J(\theta)}{\partial \theta}$ It is possible to summarize the Gradient Descent algorithm in **a single line!**

We have:

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Note: the dimensions θ are obviously preserved: $(n+1) \times 1$